



I don't know how to complete the lab

Give me the lessons and concepts, the instructions to complete, how to complete each and every instruction and why. I also need to know where to do what, all in accordance with the Lab 7 file. Be comprehensive and make your response highly comprehensible

This is a comprehensive guide to completing Lab 7 from start to finish. Everything is organised in the order you should do it, with full code, explanations of *why* each piece exists, and *where* each file lives.

Lab 7 Complete Guide

The Big Picture

You are building four Docker containers that talk to each other:^[1]

Container	Role	What you write
api	A FastAPI REST server	api/main.py
ollama	Hosts local LLMs	ollama/Dockerfile + entrypoint.sh
monitor	Agentic monitoring loop	monitor/monitor.py
nest	Jupyter notebook (client)	Notebook cells

The end result: a system where a monitor agent **polls the API**, sends metrics to an LLM for interpretation, and **decides whether to escalate** — all running locally with no internet dependency.^[1]

Phase 1 — Environment Setup

Step 1: Create the folder structure

Where: Your host machine (PowerShell/Terminal), NOT inside Docker yet.

```
mkdir lab7
cd lab7
mkdir api
mkdir monitor
mkdir ollama
```

Why: Each container is built from its own subfolder. Docker needs these folders to exist before it can build images.^[1]

Your final structure will look like:

```
lab7/
├── docker-compose.yml
├── api/
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── monitor/
│   ├── Dockerfile
│   ├── monitor.py
│   └── requirements.txt
└── ollama/
    ├── Dockerfile
    └── entrypoint.sh
```

Step 2: Create `docker-compose.yml`

Where: Inside `lab7/` (the root folder, NOT in any subfolder)

```
services:
  nest:
    image: pedrombmachado/nest:comp40771
    container_name: comp40771_container
    ports:
      - "8081:8080"
    volumes:
      - docker_comp40771:/opt/data
    environment:
      - NEST_CONTAINER_MODE=jupyterlab
      - OLLAMA_BASE=http://ollama:11434
      - API_BASE=http://api:8000
    security_opt:
      - seccomp=unconfined
    cap_add:
      - SYS_PTRACE
    depends_on:
      - ollama
      - api

  ollama:
    build: ./ollama
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama:/root/.ollama
    environment:
      - OLLAMA_MODELS=llama3.2:1b phi3:mini
```

```

api:
  build: ./api
  container_name: api
  ports:
    - "8000:8000"
  environment:
    - LOG_LEVEL=INFO
  depends_on:
    - ollama

monitor:
  build: ./monitor
  container_name: monitor
  environment:
    - API_BASE=http://api:8000
    - OLLAMA_BASE=http://ollama:11434
    - MODEL_LIGHT=llama3.2:1b
    - MODEL_HEAVY=phi3:mini
    - ESCALATION_WEBHOOK=
  depends_on:
    - api
    - ollama

volumes:
  docker_comp40771:
  ollama:

```

Why: This file defines all four services and their relationships. `depends_on` ensures containers start in the right order — for example, `api` won't start before `ollama` is up. Docker's internal DNS (`http://ollama:11434`) lets containers find each other by name.^[1]

Phase 2 — The API Container

Step 3: Create `api/requirements.txt`

Where: `lab7/api/requirements.txt`

```

fastapi==0.111.0
uvicorn==0.29.0
pydantic==2.7.1
prometheus-client==0.20.0
requests==2.32.3

```

Why: These are the Python libraries the API needs. FastAPI builds the REST server, uvicorn runs it, pydantic validates data, prometheus-client exposes metrics, and requests allows the API to call Ollama.^[1]

Step 4: Create `api/Dockerfile`

Where: `lab7/api/Dockerfile`

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY main.py .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why: This tells Docker how to build the API container image. It installs dependencies first (so rebuilds are faster if only code changes), then copies the code, and runs uvicorn as the web server on port 8000.^[1]

Step 5: Create `api/main.py`

Where: `lab7/api/main.py`

This is the **core of the REST API**. Copy the base version provided, then you will extend it in Tasks A and B.

```
import os
import time
import random
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from prometheus_client import Counter, Histogram, generate_latest, CONTENT_TYPE_LATEST
from starlette.responses import Response

app = FastAPI(title="Demo REST API", version="1.0.0")

# Prometheus metrics counters
REQ_COUNT = Counter("requests_total", "Total requests", ["endpoint", "status"])
REQ_LAT = Histogram("request_latency_seconds", "Latency", ["endpoint"])
ERR_COUNT = Counter("errors_total", "Total errors", ["endpoint", "type"])

class OrderIn(BaseModel):
    item: str
    quantity: int

orders = {}
order_id_seq = 1

def maybe_inject_failure(p: float = 0.0):
```

```

    if random.random() < p:
        raise RuntimeError("Injected failure for lab demonstration")

@app.get("/health")
def health():
    return {"status": "ok", "time": int(time.time())}

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

@app.get("/v1/orders/{order_id}")
def get_order(order_id: int):
    endpoint = "/v1/orders/{id}"
    with REQ_LAT.labels(endpoint=endpoint).time():
        order = orders.get(order_id)
        if order is None:
            REQ_COUNT.labels(endpoint=endpoint, status="404").inc()
            raise HTTPException(status_code=404, detail="Order not found")
        REQ_COUNT.labels(endpoint=endpoint, status="200").inc()
        return {"id": order_id, **order}

@app.post("/v1/orders", status_code=201)
def create_order(order: OrderIn):
    global order_id_seq
    endpoint = "/v1/orders"
    with REQ_LAT.labels(endpoint=endpoint).time():
        fail_rate = float(os.getenv("FAIL_RATE", "0.0"))
        maybe_inject_failure(fail_rate)
        if order.quantity <= 0:
            REQ_COUNT.labels(endpoint=endpoint, status="400").inc()
            raise HTTPException(status_code=400, detail="Quantity must be positive")
        order_id = order_id_seq
        order_id_seq += 1
        orders[order_id] = {"item": order.item, "quantity": order.quantity, "created_a
        REQ_COUNT.labels(endpoint=endpoint, status="201").inc()
        return {"id": order_id, **orders[order_id]}

@app.get("/v1/slow")
def slow():
    endpoint = "/v1/slow"
    with REQ_LAT.labels(endpoint=endpoint).time():
        time.sleep(random.uniform(0.2, 1.2))
        REQ_COUNT.labels(endpoint=endpoint, status="200").inc()
        return {"ok": True}

```

Why each part exists:

- `/health`— tells orchestration systems "I'm alive"^[1]
- `/metrics`— exposes Prometheus-format stats; the monitor agent reads this to know error rates and latencies^[1]
- `/v1/orders`— your main resource; uses proper noun-based URI design^[1]
- `/v1/slow`— deliberately slow endpoint used by the monitor to test latency detection^[1]

- REQ_COUNT, REQ_LAT, ERR_COUNT — track statistics that the monitor will read^[1]

Phase 3 — The Ollama Container

Step 6: Create ollama/Dockerfile

Where: lab7/ollama/Dockerfile

```
FROM ollama/ollama:latest

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

ENV OLLAMA_MODELS="llama3.2:1b phi3:mini"

ENTRYPOINT ["/entrypoint.sh"]
```

Why: The standard Ollama image just starts the server. You need a custom entrypoint that also auto-downloads the models you need. The ENV line sets which models to pull.^[1]

Step 7: Create ollama/entrypoint.sh

Where: lab7/ollama/entrypoint.sh

```
#!/bin/sh
set -eu

# Start Ollama server in the background
ollama serve &
SERVER_PID="$!"

# Wait until the server is ready
i=0
until ollama list >/dev/null 2>&1; do
    i=$((i+1))
    if [ "$i" -ge 60 ]; then
        echo "Ollama server did not become ready in time." >&2
        kill "$SERVER_PID" 2>/dev/null || true
        exit 1
    fi
    sleep 1
done

# Pull required models (fast if already cached in the volume)
for m in $OLLAMA_MODELS; do
    echo "Ensuring model is available: $m"
    ollama pull "$m"
done
```

```
# Stay running (keep server in foreground)
wait "$SERVER_PID"
```

Why: This script starts the Ollama server in the background, waits for it to be ready, pulls both models (llama3.2:1b = light/fast, phi3:mini = heavier/smarter), then keeps the container alive. The volume mount means models are only downloaded once — subsequent starts use the cache.^[1]

Phase 4 — The Monitor Container

Step 8: Create monitor/requirements.txt

Where: lab7/monitor/requirements.txt

```
requests==2.32.3
```

Why: The monitor only needs the requests library to make HTTP calls to the API and to Ollama.^[1]

Step 9: Create monitor/Dockerfile

Where: lab7/monitor/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY monitor.py .

CMD ["python", "monitor.py"]
```

Step 10: Create monitor/monitor.py

Where: lab7/monitor/monitor.py

```
import os, time, json, re, requests

API_BASE = os.getenv("API_BASE", "http://localhost:8000")
OLLAMA_BASE = os.getenv("OLLAMA_BASE", "http://localhost:11434")
MODEL_LIGHT = os.getenv("MODEL_LIGHT", "llama3.2:1b")
MODEL_HEAVY = os.getenv("MODEL_HEAVY", "phi3:mini")
ESCALATION_WEBHOOK = os.getenv("ESCALATION_WEBHOOK", "").strip()
```

```
POLL_SECONDS = 10
```

```
def fetch_metrics_text() -> str:
```

```
    r = requests.get(f"{API_BASE}/metrics", timeout=5)
    r.raise_for_status()
    return r.text
```

```
def parse_metric(metrics_text: str, name: str) -> float:
```

```
    total = 0.0
    for line in metrics_text.splitlines():
        if line.startswith("#"):
            continue
        if line.startswith(name + "{") or line.startswith(name + " "):
            parts = line.split()
            if len(parts) >= 2:
                try:
                    total += float(parts[-1])
                except ValueError:
                    pass
    return total
```

```
def ollama_generate(model: str, prompt: str) -> str:
```

```
    payload = {"model": model, "prompt": prompt, "stream": False}
    r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload, timeout=30)
    r.raise_for_status()
    return r.json().get("response", "").strip()
```

```
def triage_with_llm(model: str, context: dict) -> dict:
```

```
    prompt = (
        "You are an SRE triage assistant.\n"
        "Given the monitoring context, output STRICT JSON with keys:\n"
        "\"severity\" (low|medium|high|critical), confidence (0-1), summary, recommended_action\n"
        "No extra text.\n\n"
        f"Context:\n{json.dumps(context, indent=2)}\n"
    )
    raw = ollama_generate(model, prompt)
    m = re.search(r"\{.*\}", raw, re.DOTALL)
    if not m:
        return {"severity": "medium", "confidence": 0.3,
                "summary": "Model output not parseable.",
                "recommended_action": "Inspect metrics manually."}
    try:
        return json.loads(m.group(0))
    except json.JSONDecodeError:
        return {"severity": "medium", "confidence": 0.3,
                "summary": "JSON decode failed.",
                "recommended_action": "Inspect metrics manually."}
```

```
def should_escalate(assessment: dict, error_rate_delta: float, latency_hint: bool) -> str:
```

```
    sev = assessment.get("severity", "medium")
    conf = float(assessment.get("confidence", 0.0))
    # Deterministic guardrails – always override model
    if error_rate_delta >= 5:
        return "human"
    if latency_hint and sev in ("high", "critical"):
```



```

        return "human"
    # Confidence-based escalation
    if sev in ("high", "critical") and conf < 0.7:
        return "heavy"
    if sev == "medium" and conf < 0.5:
        return "heavy"
    return "none"

def notify_human(message: str):
    print(f"[ESCALATE:HUMAN] {message}")
    if ESCALATION_WEBHOOK:
        try:
            requests.post(ESCALATION_WEBHOOK, json={"text": message}, timeout=5)
        except Exception:
            print("[WARN] Webhook failed")

def main():
    print("[monitor] starting")
    prev_errors = None
    prev_reqs = None

    while True:
        try:
            metrics = fetch_metrics_text()
            errors_total = parse_metric(metrics, "errors_total")
            reqs_total = parse_metric(metrics, "requests_total")

            if prev_errors is None:
                prev_errors, prev_reqs = errors_total, reqs_total
                time.sleep(POLL_SECONDS)
                continue

            errors_delta = errors_total - prev_errors
            reqs_delta = max(reqs_total - prev_reqs, 0.0)
            latency_hint = False # Extended in Task C

            context = {
                "window_seconds": POLL_SECONDS,
                "requests_delta": reqs_delta,
                "errors_delta": errors_delta,
                "api_base": API_BASE
            }

            assessment_light = triage_with_llm(MODEL_LIGHT, context)
            decision = should_escalate(assessment_light, errors_delta, latency_hint)
            print(f"[monitor] light_assessment={assessment_light} decision={decision}")

            if decision == "heavy":
                assessment_heavy = triage_with_llm(MODEL_HEAVY, context)
                print(f"[monitor] heavy_assessment={assessment_heavy}")
                if assessment_heavy.get("severity") in ("high", "critical"):
                    notify_human(json.dumps(assessment_heavy))
            elif decision == "human":
                notify_human(json.dumps({"light_assessment": assessment_light, "context":

            prev_errors, prev_reqs = errors_total, reqs_total

```

```

        except Exception as e:
            print(f"[monitor] error={type(e).__name__} msg={e}")

        time.sleep(POLL_SECONDS)

if __name__ == "__main__":
    main()

```

Why each part exists:

- `fetch_metrics_text()` — **Sense:** pulls Prometheus metrics from the API^[1]
- `trriage_with_llm()` — **Reason:** sends a structured prompt to Ollama, extracts a JSON assessment^[1]
- `should_escalate()` — **Act:** first checks hard guardrails (5+ errors = always escalate), then uses LLM confidence to decide^[1]
- `notify_human()` — prints a human-readable page event; optionally POSTs to a webhook^[1]

Step 11 & 12: Build and verify

Where: `lab7/` folder on your host machine

```

# Build and start all 4 containers
docker compose up -d --build

# Confirm models were pulled into Ollama
docker exec -it ollama ollama list

```

Why: `--build` forces Docker to rebuild images from your Dockerfiles rather than using cached ones. After this, your system is running.^[1]

Phase 5 — Advanced Tasks (Assessed Coursework)

These are the tasks you must implement and commit to your private CWK repository.^[1]

Task A — REST API Enhancements (`api/main.py`)

A1: Add `PUT /v1/orders/{id}` — allows updating an existing order's quantity.

```

class OrderUpdate(BaseModel):
    quantity: int

@app.put("/v1/orders/{order_id}")
def update_order(order_id: int, update: OrderUpdate):
    if update.quantity <= 0:
        raise HTTPException(status_code=400, detail="Quantity must be positive")
    if order_id not in orders:

```

```

        raise HTTPException(status_code=404, detail="Order not found")
    orders[order_id]["quantity"] = update.quantity
    return {"id": order_id, **orders[order_id]}

```

Why: PUT is the idempotent "replace" verb. Calling it twice with the same data gives the same result, which is required REST behaviour.^[1]

A2: Add pagination GET /v1/orders — returns a slice of all orders.

```

@app.get("/v1/orders")
def list_orders(limit: int = 10, offset: int = 0):
    if limit <= 0 or offset < 0:
        raise HTTPException(status_code=400, detail="Invalid pagination parameters")
    all_ids = sorted(orders.keys())
    page = all_ids[offset: offset + limit]
    items = [{"id": oid, **orders[oid]} for oid in page]
    return {"items": items, "total": len(all_ids), "limit": limit, "offset": offset}

```

Why: Returning all records at once doesn't scale. Pagination (limit/offset) lets clients fetch manageable chunks and is standard REST practice.^[1]

A3: Standardise error responses — make all errors return the same JSON shape.

```

from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_error_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={"error": "Validation error", "detail": str(exc)}
    )

@app.exception_handler(HTTPException)
async def http_error_handler(request, exc):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": exc.detail}
    )

```

Why: Clients (and the Jupyter notebook in Task E) need to parse error responses programmatically. Consistent structure ({"error": "..."}) makes this reliable.^[1]

Task B — LLM Integration via REST (api/main.py)

B1 + B2: Add POST /v1/triage — an endpoint that takes a prompt and returns an LLM-generated triage decision.

```

import requests as http_requests

```

```

class TriageRequest(BaseModel):
    prompt: str

class TriageResponse(BaseModel):
    severity: str
    confidence: float
    summary: str
    recommended_action: str

@app.post("/v1/triage", response_model=TriageResponse)
def triage(req: TriageRequest):
    if not req.prompt or not req.prompt.strip():
        raise HTTPException(status_code=400, detail="Prompt must not be empty")

    ollama_url = "http://ollama:11434/api/generate"
    full_prompt = (
        "You are an SRE triage assistant. Output STRICT JSON only with keys: "
        "severity (low|medium|high|critical), confidence (0.0-1.0), summary, recommended_action\n"
        f"Input: {req.prompt}"
    )
    try:
        r = http_requests.post(ollama_url, json={"model": "llama3.2:1b", "prompt": full_prompt})
        r.raise_for_status()
        raw = r.json().get("response", "")
        match = re.search(r"\{.*\}", raw, re.DOTALL)
        if not match:
            raise HTTPException(status_code=502, detail="LLM returned unparseable output")
        data = json.loads(match.group(0))
        return TriageResponse(**data) # Pydantic validates the schema
    except json.JSONDecodeError:
        raise HTTPException(status_code=502, detail="LLM JSON decode failed")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

Why: This exposes the LLM as an API endpoint. The `TriageResponse` Pydantic model validates the LLM's output before returning it — if the model hallucinates a wrong field name, Pydantic rejects it and you return a 502 instead of broken JSON.^[1]

Task C — Agentic Monitoring (`monitor/monitor.py`)

C1: Detect high latency — measure how long `/v1/slow` actually takes.

Add this function and call it in `main()`:

```

LATENCY_THRESHOLD_SECONDS = 0.8

def check_latency() -> bool:
    """Returns True if latency exceeds threshold (latency_hint=True)."""
    try:
        start = time.time()
        r = requests.get(f"{API_BASE}/v1/slow", timeout=5)
        elapsed = time.time() - start
        print(f"[monitor] latency={elapsed:.3f}s")
    except:
        return False
    return elapsed > LATENCY_THRESHOLD_SECONDS

```

```

        return elapsed > LATENCY_THRESHOLD_SECONDS
    except Exception as e:
        print(f"[monitor] latency check failed: {e}")
        return False

```

Then in `main()`, replace `latency_hint = False` with:

```
latency_hint = check_latency()
```

Why: The base monitor sets `latency_hint = False` as a placeholder. Task C asks you to make it real by actually timing calls to the slow endpoint.^[1]

C2: Implement runbook actions — replace silent decisions with printed actions.

Add this function and call it after `assessment_light` is computed:

```

def run_runbook(assessment: dict):
    sev = assessment.get("severity", "low")
    summary = assessment.get("summary", "")
    action = assessment.get("recommended_action", "")
    if sev == "low":
        print(f"[RUNBOOK:LOG] severity=low | {summary}")
    elif sev == "medium":
        print(f"[RUNBOOK:TICKET] Creating ticket – {summary} | action={action}")
    elif sev in ("high", "critical"):
        print(f"[RUNBOOK:PAGE] ☒ Paging on-call – {summary} | action={action}")

```

Call it in `main()` after getting `assessment_light`:

```
run_runbook(assessment_light)
```

Why: Real agentic systems don't just classify — they *do something* based on the classification. This implements the "Act" phase of the Sense-Reason-Act loop.^[1]

Task D — Adaptive Escalation (`monitor/monitor.py`)

D1: Confidence floor rule — already partially in `should_escalate()`, but make it explicit:

```

CONFIDENCE_FLOOR = 0.7

def should_escalate(assessment: dict, error_rate_delta: float, latency_hint: bool) -> str:
    sev = assessment.get("severity", "medium")
    conf = float(assessment.get("confidence", 0.0))
    # Hard guardrails always win
    if error_rate_delta >= 5:
        return "human"
    if latency_hint and sev in ("high", "critical"):
        return "human"
    # Low confidence = escalate to heavy model for second opinion

```

```

if sev in ("high", "critical") and conf < CONFIDENCE_FLOOR:
    return "heavy"
if sev == "medium" and conf < 0.5:
    return "heavy"
return "none"

```

D2: Persistence logic — escalate if medium alert persists for N consecutive cycles.

Add a counter at the module level and update `main()`:

```

# At module level
medium_counter = 0
MEDIUM_PERSIST_THRESHOLD = 3 # escalate after 3 consecutive medium alerts

# Inside main() loop, after getting assessment_light:
global medium_counter
sev = assessment_light.get("severity", "low")

if sev == "medium":
    medium_counter += 1
    if medium_counter >= MEDIUM_PERSIST_THRESHOLD:
        print(f"[monitor] Medium alert persisted {medium_counter} cycles – escalating to high")
        assessment_light["severity"] = "high"
        medium_counter = 0 # reset after escalating
    else:
        medium_counter = 0 # reset on any non-medium result

```

Why: A single medium alert could be a transient blip. Persistence logic prevents false alarms while ensuring genuine sustained problems get escalated — this is "anti-flapping".^[1]

Task E — Jupyter Notebook (nest container, <http://localhost:8081/lab>)

Where: In the Jupyter notebook `comp40771_lab7.ipynb`, accessed via browser at <http://localhost:8081/lab>

Cell 1 — Configuration and helpers:

```

import requests, json

API_BASE = "http://api:8000"

def show(title, response):
    print(f"\n== {title} ==")
    print(f"Status: {response.status_code}")
    try:
        print(json.dumps(response.json(), indent=2))
    except Exception:
        print(response.text)

def create_order(item, quantity):
    return requests.post(f"{API_BASE}/v1/orders", json={"item": item, "quantity": quant

```

```
def get_orders(limit=10, offset=0):
    return requests.get(f"{API_BASE}/v1/orders", params={"limit": limit, "offset": offset})

def update_order(order_id, quantity):
    return requests.put(f"{API_BASE}/v1/orders/{order_id}", json={"quantity": quantity})

def triage(prompt):
    return requests.post(f"{API_BASE}/v1/triage", json={"prompt": prompt})
```

Cell 2 — Valid calls:

```
show("Health check", requests.get(f"{API_BASE}/health"))
show("Create order", create_order("book", 3))
show("Create another order", create_order("laptop", 1))
show("List orders (paginated)", get_orders(limit=2, offset=0))
show("Update order 1", update_order(1, 10))
show("Triage prompt", triage("Error rate spiked to 15% after deployment"))
```

Cell 3 — Invalid calls (error code verification):

```
show("POST with quantity=0 (expect 400)", create_order("pen", 0))
show("PUT with quantity=-1 (expect 400)", update_order(1, -1))
show("GET non-existent order (expect 404)", requests.get(f"{API_BASE}/v1/orders/999999"))
show("GET with limit=-1 (expect 400)", get_orders(limit=-1, offset=0))
show("POST triage empty prompt (expect 400)", triage(""))
```

Why all of this: Task E proves your entire system works end-to-end. You're acting as a client testing the API you built, verifying both happy paths (201, 200) and error paths (400, 404).^[1]

Phase 6 — Committing Your Work

Where: Jupyter terminal at <http://localhost:8081/lab>

```
cd /opt/data/comp40771_lab7

git config --global user.name "Your Name"
git config --global user.email "nXXXXXX@ntu.ac.uk"

git add *
git commit -m "Lab 7 complete: REST API, LLM triage, agentic monitoring, adaptive escalation"
git push
```

Why: This is assessed coursework. Your solution must be pushed to your private CWK repository on Olympus to be marked.^[1]

Quick Reference: What File Lives Where

File	Location	What it does
<code>docker-compose.yml</code>	<code>lab7/</code>	Orchestrates all 4 containers
<code>api/main.py</code>	<code>lab7/api/</code>	REST API — Tasks A & B
<code>api/Dockerfile</code>	<code>lab7/api/</code>	Builds the API container
<code>api/requirements.txt</code>	<code>lab7/api/</code>	Python deps for API
<code>monitor/monitor.py</code>	<code>lab7/monitor/</code>	Agentic loop — Tasks C & D
<code>monitor/Dockerfile</code>	<code>lab7/monitor/</code>	Builds monitor container
<code>monitor/requirements.txt</code>	<code>lab7/monitor/</code>	Python deps for monitor
<code>ollama/Dockerfile</code>	<code>lab7/ollama/</code>	Builds LLM container
<code>ollama/entrypoint.sh</code>	<code>lab7/ollama/</code>	Auto-pulls models on start
<code>comp40771_lab7.ipynb</code>	Jupyter (/opt/data/)	Task E notebook



1. Lab-7REST-APIs-Local-Language-Models-Agentic-Monitoring-and-Adaptive-Escalation.pdf